L03 — Deep Learning Preliminaries

Neural nets, RNNs, attention, and Transformers

Study summary generated from lecture slides. ETH AI for Digital Characters.

Neural network unit

- $z = w^*x + b$; output $a = \text{sigma}(z)$
- Activations: ReLU (most common), sigmoid [0,1], tanh [-1,1]
- Softmax: converts logits to probabilities summing to 1
- Training: forward pass -> loss (cross-entropy) -> backpropagation

Feedforward networks

- Input -> hidden (ReLU) -> output (sigmoid or softmax)
- Applications: sentiment classification, language modeling

Recurrent Neural Networks

- h_t = sigma(U*h_{t-1} + W*x_t); y_t = f(V*h_t)
- Shared weights U, V, W across time steps; processed sequentially
- Captures sequential data via hidden state memory
- Vanishing gradient problem limits long-range dependencies
- LSTM: forget, input, output gates + cell state highway
- Bidirectional RNN: left-to-right + right-to-left, concatenate hidden states
- Stacked RNNs: multiple layers for richer representations
- Uses: language modeling, sentiment classification (last hidden state), generation
- Teacher forcing: use gold previous token during training

Encoder-decoder

- Encoder summarizes input (x1…xT) into context vector h_T
- Decoder generates output sequence autoregressively
- Applications: machine translation, summarization, dialogue
- p(y|x) = product of conditional next-token probabilities

Attention mechanism

- Problem: RNN bottleneck and vanishing gradients for long sequences
- Score: sim(q, k_t) = q*k_t / sqrt(d) [scaled dot-product]
- Weights: a_t = softmax(scores); Context: c = sum a_t * v_t
- Self-attention: Q, K, V from same sequence
- Cross-attention: Q from decoder, K/V from encoder
- Multi-head attention: multiple parallel attention heads

Transformers

- No inherent order -> positional encoding added to embeddings
- $PE(t,2i) = \sin(t/10000^{(2i/d)})$; $PE(t,2i+1) = \cos(t/10000^{(2i/d)})$
- Encoder: multi-head self-attention + FFN + skip connections + layer norm
- Decoder: masked self-attention (no future peeking) + cross-attention
- Layer norm: normalize across sequence per example (not batch norm)